HuskyPark Predictor

AI-Driven Parking Spot Probability Dashboard for SCSU

Web Interface Design Report: Phase 3

Course

CSCI 414/514: Platform Based Development

Section

Group 8

Members

Chidubem Okoye, Yvonne Onmakpo, Tonika Devi Avanigadda

Date

March 2026

Institution

St. Cloud State University

Abstract

This report presents the web interface design for HuskyPark Predictor, a probability based parking dashboard tailored to St. Cloud State University. It details all planned user-facing features, CRUD operations, search and filter strategies, interface layout, navigation structure, backend integration patterns, and usability justifications. Three original diagrams are included: a navigation flow diagram, a system data-flow diagram, and a user journey flowchart.

A. Group Information

Field	Detail
Project Title	HuskyPark Predictor: AI-Driven Parking Spot Probability Dashboard for SCSU
Course	CSCI 414/514; Platform Based Development
Section	Group 8
Members	Chidubem Okoye, Yvonne Onmakpo, Tonika Devi Avanigadda
Date	March 2026
Institution	St. Cloud State University

B. Functional Design

B.1 Overview of Planned Features

The HuskyPark Predictor web interface is organized around four primary roles: Student/Commuter, Faculty/Staff, Visitor, and Administrator. Each role interacts with a tailored subset of features. The features below cover all required CRUD operations as well as advanced capabilities such as AI powered recommendations, data filtering, and role-specific dashboards.

Feature	User Actions	Backend Connection
Lot Availability Dashboard	View real time probability scores per lot by permit type, filter by category or time, sort by availability	GET /api/dashboard → vw_latest_predictions (PostgreSQL) + lot_hourly_analytics (MongoDB)
Spot Report Submission (Create)	Select a lot, submit status (found_spot / lot_full / hard_to_find), optionally attach estimated available count or photo	POST /api/reports → CrowdsourcedReport (SQL) + crowdsourced_reports (MongoDB)
AI Recommendation Engine	Enter natural-language query (e.g., 'Park near Atwood at 9 AM'), receive ranked lot suggestions with rationale	POST /api/recommend → LLM prompt + vw_latest_predictions + WeatherSnapshot → ai_recommendation_sessions (MongoDB)
Permit Management (CRUD)	Students/staff view, create, update, or deactivate their parking permit records	GET/POST/PATCH /api/permits → Permit table (PostgreSQL)
Campus Event Calendar (Admin CRUD)	Admins add, edit, or delete campus events and assign lot impact multipliers	GET/POST/PUT/DELETE /api/events → CampusEvent + EventLotImpact (PostgreSQL)
User Account Management (Admin)	Admins create, search, update, or deactivate user accounts; filter by type/status	GET/POST/PATCH /api/users → User table (PostgreSQL)
Lot & Zone Management (Admin)	Admins add new lots, edit capacity or GPS coordinates, manage sub-zones	GET/POST/PUT /api/lots and /api/zones → ParkingLot + ParkingZone (PostgreSQL)
Crowd Report History (Search & Filter)	View crowd report history per lot; filter by date range, status, or source type	GET /api/reports?lot_id=&status=&from=&to= → CrowdsourcedReport (PostgreSQL)
Analytics & Trends Panel	View hourly parking trends per lot for the last 7 or 30 days; visualize probability history	GET /api/analytics → lot_hourly_analytics (MongoDB)
Prediction Explanation Modal	Click a lot to expand JSONB factor breakdown (weather, event impact, recent reports)	GET /api/predictions/:id → factors_summary JSONB (PostgreSQL)
Notifications / Alerts	Receive browser-push or in-app alerts when a monitored lot drops below a probability threshold	Background scheduler triggers → notification_logs (MongoDB) → frontend push
Data Export (Admin)	Download CSV of prediction	GET /api/export?type=&from=&to= →

	history, crowd reports, or permit records for a chosen date range	PostgreSQL query → streamed CSV response

B.2 CRUD Operations in Detail

B.2.1 Create Operations

New records are created through form based interactions across multiple modules.

- Spot Report: Any authenticated user can submit a spot status for any lot. The form collects lot selection (dropdown), status (radio group), and an optional spot count. After submission a success toast confirms the write, and the dashboard probability updates within 60 seconds via the background prediction recalculation job.
- Permit: A user fills out a permit application form with lot preference, permit type, vehicle plate, and date range. The backend validates that the end date is after the start date and that the permit type matches the lot category before inserting into the Permit table.
- Campus Event (Admin): The admin event form collects event name, date, time window, venue, estimated attendance, impact level, and the lots affected with their demand multipliers. Validation ensures no two high-impact events share the same date without review.
- Lot / Zone (Admin): Admins use a map enabled form to place a new lot marker, fill in capacity, permit type, and lot type. A sub form allows adding parking zones (row labels, zone type, capacity) under the lot.

B.2.2 Read / Search Operations

All data views are read operations. Key patterns include:

- Dashboard Load: On page load the frontend calls GET /api/dashboard, which returns the merged PostgreSQL prediction and MongoDB analytics payload for all lots. Results are rendered as probability cards with color indicators (green >= 0.65, yellow 0.35–0.64, red < 0.35).
- Lot Detail Drill-Down: Clicking a lot card triggers GET /api/lots/:id/predictions, fetching the 24-hour prediction history and the latest crowd reports for that lot. The factors_summary JSONB is expanded into a human-readable breakdown modal.
- Admin Search: The Users and Permits admin panels expose a full-text search bar. Queries are forwarded as GET /api/users?q=&type=&active= and run against the email and full_name columns in PostgreSQL using ILIKE for case-insensitive matching.

B.2.3 Update Operations

Updates are performed via forms pre populated with existing data.

- Permit Update: Users can modify their permit's vehicle plate or extend the end date. Partial updates use PATCH /api/permits/:id. The backend enforces that the end date cannot be set to a date in the past.
- Event Edit (Admin): Admins can modify event details or adjust lot impact multipliers. Updating a demand multiplier triggers an asynchronous prediction recalculation for all affected lots.
- User Account Update (Admin): Admins can toggle the is_active flag to deactivate accounts without deleting them, or update user type. Changes are audited in a change log.
- Lot Capacity Update (Admin): If a construction project temporarily closes a zone, admins adjust zone capacity. The prediction engine incorporates the updated capacity on the next cycle.

B.2.4 Delete Operations

All delete actions in HuskyPark use soft deletion or are constrained by referential integrity to preserve historical data.

- Spot Report Soft Delete: Reports submitted in error can be flagged as invalid by admins; they are not physically deleted but excluded from aggregation queries via a status='invalid' filter.
- Campus Event Delete (Admin): Deleting a campus event cascades to EventLotImpact rows (via ON DELETE CASCADE). A confirmation dialog shows all affected lots before the admin confirms.
- Permit Deactivation: Permits cannot be hard-deleted because they anchor financial and audit records. Instead, the end_date is backdated to today, effectively deactivating the permit while preserving the record.
- User Deactivation: Admins toggle is_active = FALSE. The user cannot log in but all associated reports and permits remain intact.

B.3 Search and Filtering Strategy

The interface exposes structured filtering on all major data views to reduce the cognitive load of browsing large datasets.

Panel	Available Filters
Dashboard	Permit type (COMMUTER / RESIDENT / EMPLOYEE / PAY), lot name search, sort by probability (high or low), hide lots with no active permit match
Crowd Report History	Lot (multi-select), status (found_spot / lot_full / hard_to_find), date range picker, source type (user / simulated), minimum approx_available count
Prediction History	Lot, model version, confidence level (low / medium / high), date range, weather condition (from factors_summary JSONB)
Admin Users	User type, active status, created date range, free-text search on name or email
Admin Permits	Permit type, lot, active only toggle (end_date >= today), vehicle plate search
Campus Events	Date range, impact level (low / medium / high), affected lot filter
Analytics Trends	Lot, day of week, time range (last 7 / 14 / 30 days), weather condition overlay

All filter parameters are forwarded as query string parameters to the backend REST endpoints and mapped to WHERE clause predicates in the PostgreSQL queries or to MongoDB query documents. JSONB-level filters use the GIN index created in Phase 2 to remain performant.

C. Interface Layout & Navigation Design

C.1 Main Pages

The application is structured around six primary page groups. Navigation is handled through a persistent left sidebar that collapses to an icon-only view on smaller screens, with a top bar for user profile, role indicator, and notifications.

Page / Module	Purpose & Key Components
1. Public Landing Page	Unauthenticated users see a read-only lot status summary (color-coded probability tiles) and a login/register call-to-action.
2. Dashboard (Home)	The primary view after login. Displays probability cards for all lots matching the user's permit type. Cards show lot name, probability score, confidence badge, last updated time, and a mini trend sparkline.
3. Lot Detail Page	Clicking any lot card opens a detail page with a 24-hour probability chart, the factors breakdown modal, recent crowd reports timeline, and a Submit Spot Report button.
4. My Permits & Profile	Users manage their own permit records: view active/expired permits, apply for a new permit, update vehicle plate, and deactivate a permit.
5. AI Recommendation Chat	A conversational panel where users type parking queries. Shows ranked lot recommendations with probability scores, distances, and AI-generated rationale.
6. Admin Panel	Role-gated section with sub-pages for: User Management, Lot & Zone Management, Campus Event Management, Prediction Configuration, and Data Export.
7. Analytics & Reports	Accessible to admins and faculty/staff. Shows hourly trend charts per lot, 7-day and 30-day rolling averages, and weather correlation charts sourced from MongoDB lot_hourly_analytics.

C.2 Navigation Flow Diagram

Figure 1 illustrates the full page hierarchy and navigation paths available to authenticated users, from the landing page through to admin sub-panels.

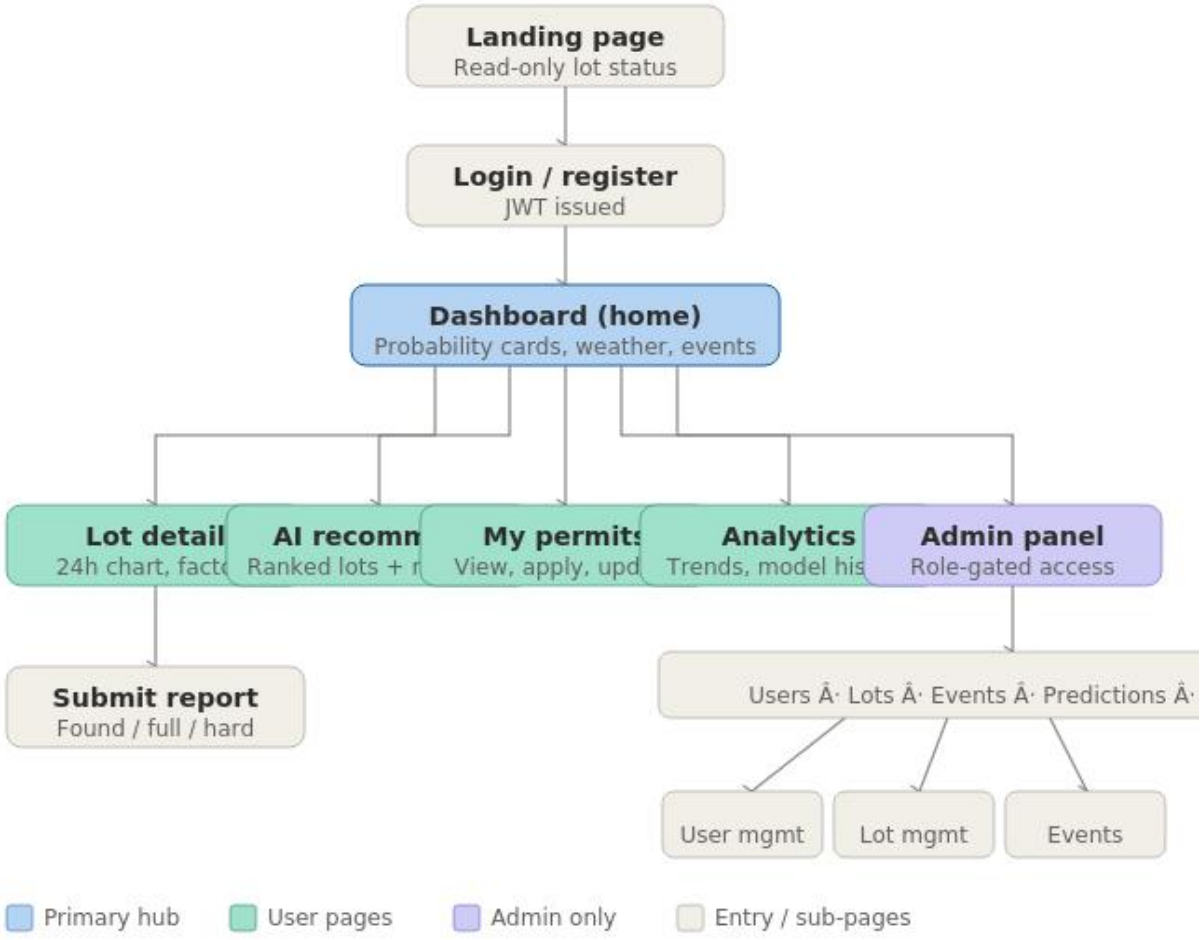

Figure 1. HuskyPark Predictor – Navigation Flow Diagram

C.3 Navigation Structure

The navigation is organized in a three-tier hierarchy to minimize clicks for the most common workflows.

- Tier 1; Global Sidebar: Dashboard, AI Recommend, My Permits, Analytics, Admin Panel (visible only to admins). The active item is highlighted with a NAVY background and gold left border.
- Tier 2; Top Bar: Search icon (opens a global lot/report search overlay), notification bell with unread badge, weather widget showing current St. Cloud conditions, and user avatar dropdown.
- Tier 3; In Page Tabs: Within the Admin Panel, a horizontal tab bar separates Users, Lots, Events, Predictions, and Export. Within Analytics, tabs separate Hourly View, Trend View, and Weather Correlation.

C.4 Key User Journeys

C.4.1 Commuter Checks Parking Before Driving to Campus

- Landing page → Login → Dashboard (default filter: COMMUTER lots)
- User scans probability cards; Lot C shows 38% (yellow). User clicks card.

- Lot Detail page opens. 24 hour chart shows availability typically improves after 9:30 AM.
- User opens AI Recommend panel and types 'Park near Atwood at 9:30 AM.' System returns ranked recommendations.

C.4.2 User Submits a Spot Report on Arrival

- From Dashboard, user taps the floating '+' report button or the 'Report Spot' button on a lot card.
- A modal opens with the lot pre-selected (GPS-assisted). User selects 'Found Spot,' enters estimated available count, and submits.
- Success toast confirms submission. Dashboard probability updates within 60 seconds.

C.4.3 Admin Adds Commencement Event

- Admin navigates to Admin Panel → Campus Events tab.
- Clicks 'New Event.' A form opens with all required fields. Admin selects affected lots with demand multipliers and saves.
- System triggers async recalculation of predictions for all affected lots. Admin sees a 'Recalculating…' status badge.

C.5 Form Design Principles

- Labels are displayed above inputs, not as placeholder text, so the field purpose remains visible after the user starts typing.
- Required fields are marked with a red asterisk (*). Optional fields are labeled '(optional)' in light gray.
- Date fields use a calendar date-picker. Date range pickers enforce that the end date cannot precede the start date.
- Dropdowns constrained to enum values (lot_type, status, impact_level) prevent invalid data from reaching the API.
- Submit buttons are disabled until all required fields pass client-side validation. Inline validation messages appear below each field on blur.
- Confirmation dialogs appear before any delete or deactivation action, stating the specific record affected and the consequence.

D. Backend Integration Plan

D.1 Frontend–Backend Communication

The React.js frontend communicates with the backend exclusively through a RESTful HTTP API served by FastAPI (Python). All requests and responses use JSON. Authentication is handled via JWT tokens stored in secure, HttpOnly cookies. The frontend never stores session tokens in localStorage to mitigate XSS risk.

- Base URL: /api/v1
- Authentication: POST /api/v1/auth/login returns a JWT. The frontend attaches the token as a Bearer header on all subsequent requests.
- Error responses follow RFC 7807 Problem Detail format with a 'status', 'title', and 'detail' field.
- CORS: The API allows requests only from the frontend origin (huskypark.scsu.edu) in production.

D.2 System Data Flow Diagram

Figure 2 shows how data flows through all system layers: from the React dashboard, through the FastAPI backend, into PostgreSQL and MongoDB, through background Celery workers, and back through the Redis cache.

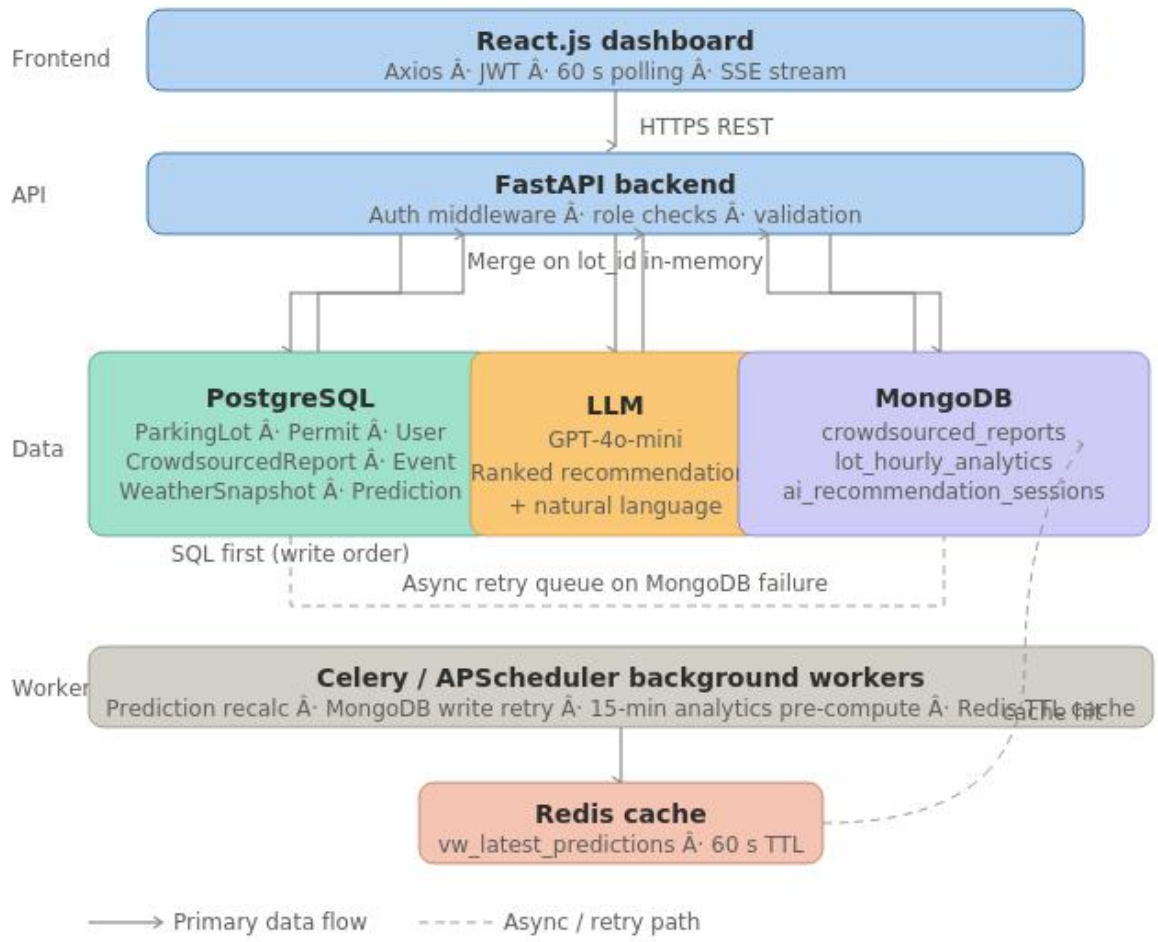

Figure 2. HuskyPark Predictor – System Data Flow Diagram

D.3 User Journey Flowchart

Figure 3 traces two core user flows at the step by step level; a spot report submission (including the decision branch for lot validation) and a dashboard load, showing how each step maps to a frontend action, API call, and database operation.

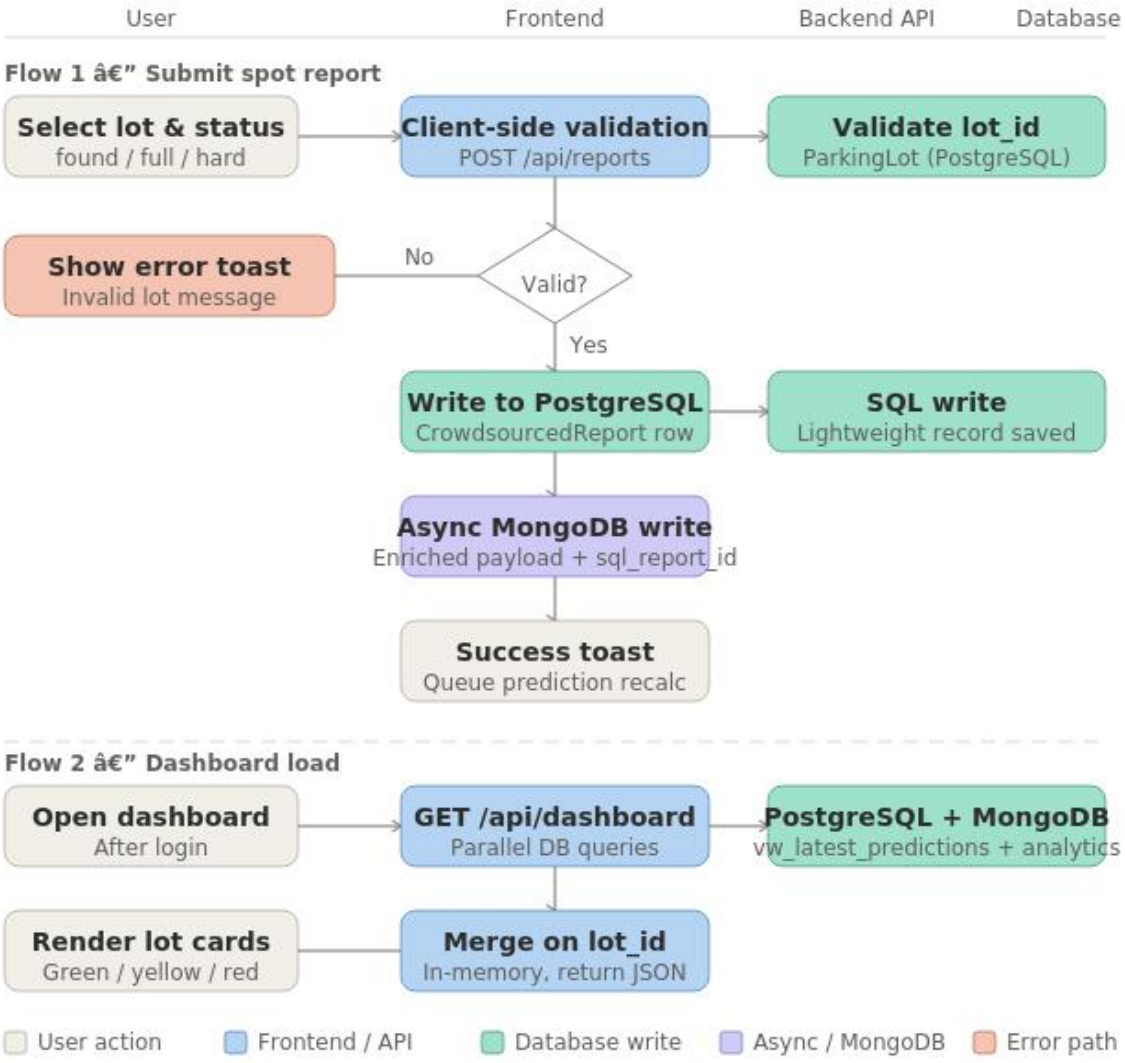

Figure 3. HuskyPark Predictor – User Journey Flowchart (Flow 1: Submit Report; Flow 2: Dashboard Load)

D.4 CRUD to Database Mapping

HTTP Method & Endpoint	Operation	Database Target
GET /api/v1/dashboard	Read merged lot predictions + analytics	PostgreSQL: vw_latest_predictions | MongoDB: lot_hourly_analytics
GET /api/v1/lots/:id/predictions	Read 24h prediction history + factors	PostgreSQL: ParkingPrediction + factors_summary JSONB

POST /api/v1/reports	Create crowd report	PostgreSQL: CrowdsourcedReport (first) | MongoDB: crowdsourced_reports (async)
GET /api/v1/reports	Read / filter crowd reports	PostgreSQL: CrowdsourcedReport (with filters)
POST /api/v1/permits	Create permit	PostgreSQL: Permit
PATCH /api/v1/permits/:id	Update permit (plate / dates)	PostgreSQL: Permit
POST /api/v1/events	Create campus event + lot impacts	PostgreSQL: CampusEvent + EventLotImpact
PUT /api/v1/events/:id	Update event details or lot multipliers	PostgreSQL: CampusEvent + EventLotImpact
DELETE /api/v1/events/:id	Delete event (cascades to EventLotImpact)	PostgreSQL: CampusEvent (CASCADE to EventLotImpact)
POST /api/v1/recommend	AI recommendation query	PostgreSQL: vw_latest_predictions + WeatherSnapshot | LLM | MongoDB: ai_recommendation_sessions
GET /api/v1/analytics	Read lot hourly analytics trends	MongoDB: lot_hourly_analytics
GET /api/v1/export	Export CSV of prediction/report data	PostgreSQL: ParkingPrediction or CrowdsourcedReport (streamed)
GET/POST/PATCH /api/v1/users	Admin user management	PostgreSQL: User
GET/POST/PUT /api/v1/lots	Admin lot management	PostgreSQL: ParkingLot + ParkingZone

D.5 Search and Filter Query Processing

- PostgreSQL: Filters map to WHERE clause predicates using parameterized queries (asyncpg). Multi-value filters expand to WHERE status = ANY($1). JSONB filters use WHERE factors_summary->>'weather' = :condition leveraging the GIN index for performance.
- MongoDB: Filter objects are built dynamically and passed to collection.find(). The lot_id and reported_at compound index handles the most common access pattern. The TTL index on crowdsourced_reports silently ages out documents older than 90 days.
- Pagination: All list endpoints return a maximum of 50 records per page. Responses include a total_count field used to render the pagination control.

D.6 Data Visualization: Retrieval and Display

- Dashboard Probability Cards: Refreshed on a 60-second polling interval. Color coding (green / yellow / red) is computed client-side from the prob_score field.
- 24-Hour Prediction Chart: GET /api/v1/lots/:id/predictions?hours=24 renders a line chart with prob_score on the y-axis and target_time on the x-axis. Weather condition overlays appear at relevant time ticks.
- Hourly Trend Charts: GET /api/v1/analytics?lot_id=&days=7 returns lot_hourly_analytics documents. The chart renders avg_prob_available_pct per hour-of-day averaged across the requested days.
- Factors Breakdown Modal: Renders the factors_summary JSONB as a structured card with labeled sections (Weather, Time of Day, Active Events, Recent Reports).

E. Usability & Design Justification

E.1 Simplicity and Clarity

HuskyPark is used most often by commuters checking lot availability before or during a drive to campus, often on a mobile device with limited time. Every design decision prioritizes getting the probability answer in as few interactions as possible.

- The dashboard is the default post-login landing page and shows all relevant lots immediately, without requiring the user to select a lot first.
- Color coding (green / yellow / red) communicates the probability score without requiring users to interpret decimals. The thresholds (>= 0.65, 0.35–0.64, < 0.35) map to the intuitive categories of likely available, uncertain, and likely full.
- The floating '+' report button is always visible on the dashboard so users can submit reports in one tap without navigating away from the lot overview.
- The sidebar navigation labels are paired with icons so frequent users can navigate by icon alone once familiar with the layout.

E.2 Role-Based Access

- Students and commuters see only the lots relevant to their permit type by default. The filter can be expanded to view all lots.
- Faculty and staff see employee lots by default and can also access the Analytics panel with read-only access to trend data.
- Visitors see only pay lots (4th Ave Ramp, Husky Lot, Eastman, ISELF, Miller, South) and cannot submit permits.
- Admins see all lots and have the Admin Panel sidebar item. All destructive actions are gated behind confirmation dialogs and role checks at the API level, not just the UI level.

E.3 Accessibility Considerations

- All interactive elements have explicit ARIA labels and are keyboard-navigable (Tab / Enter / Escape).
- Color coding alone does not convey state; each probability card also shows a text label (Available / Limited / Full) to support color-blind users.
- Font sizes use relative units (rem) so users who set a larger base font size in their browser receive proportionally larger text throughout the app.
- Minimum touch target size of 44x44 px on all interactive elements ensures usability on mobile devices.
- All charts include a data table fallback accessible via a 'View Data' toggle for screen reader users.

E.4 Error Handling and User Feedback

- Client-side validation fires on field blur and provides specific, actionable messages (e.g., 'End date must be after start date' rather than 'Invalid input').
- Failed API calls surface a non-blocking toast notification with a Retry button. The dashboard continues to display the last successfully loaded data rather than showing a blank state.

- When the MongoDB analytics layer is temporarily unavailable, the dashboard renders prediction scores from PostgreSQL with a subtle banner: 'Trend data temporarily unavailable. Prediction scores are current.' This reflects the graceful degradation architecture from Phase 2.
- Form submission states use a loading spinner on the submit button to prevent double submission.
- Long-running operations display a background status indicator so admins know the system is working.

E.5 Performance Considerations

- The 60 second dashboard polling interval balances data freshness with server load. During peak hours (8–9 AM), the Redis cache absorbs the burst of simultaneous dashboard loads by serving a cached vw_latest_predictions result with a 60 second TTL.
- Lazy loading is used for the Analytics and Admin Panel pages, keeping initial dashboard load times low.
- The Lot Detail chart fetches only 24 hours of prediction history by default. Users can request 7-day history via a toggle, which triggers an additional API call only on demand.
- Images in crowd reports are served from a CDN (cdn.huskypark.scsu.edu) to offload bandwidth from the application server.
- The AI Recommendation endpoint streams the LLM response using SSE (Server-Sent Events) so users see the rationale appear progressively rather than waiting for the full response.

E.6 Design Rationale Summary

- Speed over depth: The dashboard surfaces the most actionable data (probability score, color, lot name) at the top of the visual hierarchy. Users who need more detail drill down; users who only need a quick check get their answer without scrolling.
- Trust through transparency: The factors breakdown modal and the prediction confidence badge allow users to understand why a prediction was generated, building confidence in the system and making the AI recommendation feel grounded rather than opaque.
- Graceful degradation over hard failure: The design treats MongoDB unavailability as a recoverable condition, consistent with the write-order and retry architecture from Phase 2. The system remains useful even when the enriched analytics layer is temporarily offline.

Grading Rubric Reference

Criteria	Points
Functional completeness & clarity	25
Interface layout & navigation design	20
Backend integration explanation	20
Usability & design justification	15
Quality of diagrams/wireframes	10
Report organization & clarity	10
Total	**100**